Last-Minute Exam Kit: Reusable Megaprompt

Build a full exam-prep kit from any book plus any MCQ bank: weighted notes PDF, quiz PDF with answer key, and a quiz app with spaced repetition. Change the subject, language or sources by editing one block.

Made from a real run (Bengali ICT, 6 chapters, 375 board MCQs). Reference code is in this repo: `ict-exam-kit/scripts` and `ict-exam-kit/HANDOFF.md`.

1. Before you start (do this first)

Provide

- The book as a PDF (any language).
- The MCQ bank as PDF, slides or images, with the answers marked somewhere.
- The facts below.

Facts you must know

Item	Example
Subject and level	ICT, Class 9-10
Output language	Bengali (keep English terms as the book prints them)
Exam size and time	25 MCQs, 2 hours study
Chapters and difficulty	6 chapters, difficulty 1 to 3 each
How answers are marked in the MCQ file	red box, white dot, bold option, or key page
How chapters split in the MCQ file	title slides, headings, slide ranges
Budget	e.g. "$20 left, stop at $15"
Where to export	GitHub repo and branch

Tools needed in the session: bash, python3, poppler (`pdftotext`, `pdftoppm`), node + playwright with chromium, Pillow. For non-Latin scripts, get the font from npm (`@fontsource/...`) because CDNs may be blocked.

Quick probe (costs almost nothing)

```
pdftotext -f 20 -l 20 book.pdf - | head -20
```

If the text is clean, you do not need OCR. This is the biggest saving.

2. Token rules (read before spending anything)

The first run burned most of its credits on a swarm of Opus agents. Do not repeat that.

1. **No Opus for OCR, extraction, splitting or tagging.** Use the PDF text layer first. If OCR is needed, use Haiku.
2. **Sonnet builds chapters. Opus is allowed only for one short final audit** of items already flagged (answer-key conflicts). Never whole chapters.
3. **Max 6 agents at once, one per chapter.** Check the concurrent limit first. Stop any agent that runs long.

4. **Give agents file paths, not pasted text.** Put the rules in one instruction file and tell the agent to read it. Ask for a reply of 5 lines or less.
5. **Look at images only to settle doubts**: low-confidence answers, conflicting markers, and 10 random pages for the OCR check. Never view every page.
6. **Render small.** JPEG about 1000 px wide. Crop page halves only when text is unreadable.
7. **Use scripts for anything mechanical**: answer markers (pixel check), chapter split, weights, JSON merge, PDF build, tests. A script costs nothing to rerun.
8. **Data lives in files** (one JSON per chapter). Never paste big data into chat.
9. **Review only what is flagged**, not every chapter. One Sonnet reviewer for all flags.
10. **Test the app on mock data first**, then build with real data once.
11. **Export early**: commit and push after each stage, and keep a `HANDOFF.md`. If credits run out you can continue elsewhere.
12. **Report cost after each stage.** Stop and ask when 70% of the budget is used.

3. Pipeline

Step	Who	Output	Token tip
0 Probe and setup	main	tools OK, text layer yes/no	3 commands only
1 Book text	script, else Haiku	`book/pNNN.txt`	text layer first
2 MCQ extraction	script or Haiku, 30 slides per agent	`mcq/*.jsonl`	answer by pixel check, not vision
3 Merge, split by chapter, weights	script	`ch/mcq_chN.json`, weights	no model
4 Chapter build	Sonnet, 1 per chapter	`chN.json`	agent reads instruction file
5 Flag review	Sonnet, 1 agent, then Opus only if still undecided	fixes + `UNSURE` marks	flagged items only
6 Build app and PDFs	script	app HTML, notes PDF, quiz PDF	no model
7 Test	script (playwright)	pass/fail log	no model
8 Publish and export	main	artifact link, pushed repo	one publish

Weights: time share of a chapter = MCQ count x difficulty (1 to 3), divided by the total. Show a table and the reason.

4. Accuracy rules

- Every fact comes from the book, with a page number. Say which printed page (not the PDF index).
- Never invent. If unsure, write `UNSURE` and say why.
- State each fact with context: "true when X, false when Y".
- If an answer key looks wrong or the book disagrees, flag it. Show what the key says and what the book says.
- Check OCR: view 10 random pages against the text. Fix broken conjuncts, numbers and look-alike words.

- Confirm answers by the book, not by the key. For code questions, run the code.
- Marker conflicts are common (teacher pen marks vs the real marker). Use the real marker first, then the book.
- Report the real MCQ count. It may differ from what the user believes.
- Never use an em dash.

5. Data format (one file per chapter)

```
{ "ch": N, "title": "...", "pages": "...",
  "mcq_total": n, "top_topics": [...10], "traps_summary": [...],
  "notes": { "sections": [ { "h": "...", "items": [
        {"f": "fact line", "c": "true when X, false when Y", "p": "page", "star": true},
        {"trap": "A vs B: how to tell", "p": "page"} ] } ],
      "must": ["5 lines"] },
  "cards": [ {"id","r":1,"q","o":[4],"a":0-3,"key","b":"board","s":stars,"st":true,
              "t":"topic","k":"def|full|multi|odd|num|who|code|use|scen|fact",
              "l":"lesson 1-2 lines","p":"page","f":"flag"} ],
  "new_cards": [ same, "r":0 ],
  "flags": [...], "coverage": {"unsupported": []}, "spot_check": [...], "word_count": n }
```

Rules for notes: dense, no fluff, book wording, star facts that repeat in MCQs, add trap lines, end each chapter with a 5-line "must remember" box. Every MCQ answer must be supported by a note. Reading time = words / 150 per minute (adjust per language). If over the limit, tighten without dropping tested facts.

6. App spec

- Single HTML file, UI fully in the target language, embedded font (base64), light and dark with a toggle, works at 400 px width.
- One card at a time, large text, full-width Next button. After an answer: correct option, a "lesson" (1 to 2 lines, with page number, plus the answer pattern).
- **Review rule (important)**: a correct answer is mastered and never shown again. A wrong answer returns after 3 cards. Once the user answers it correctly, it is done. Optional override row: Again, Hard, Good, Easy (Hard = back in 8).
- **Question count selector (10 to 500)**: pick the best N cards. Split N across chapters by weight (at least 1 each). Inside a chapter, cycle through topics (most repeated first), board and starred cards first.
- Modes: mix (weighted random, +25% for starred), by chapter (next chapter button), wrong only, starred only, morning 15 minutes.
- Settings: theme, 13 accent colors, text size, shuffle questions and options, answer-pattern hints, timer, starred only, chapter filter.
- Progress: per-chapter mastery bar, time budget, weak spots list, study plan.
- Confetti at 100% chapter mastery and at session end (respect reduced motion).
- Save progress in `localStorage` inside try/catch.
- Test with playwright: no console errors, no sideways scroll, theme toggle, review rule, count selector, reload keeps progress.

7. PDF spec

- Notes PDF: weighted order (heaviest chapter first), embedded font, page numbers, weight table up front.
- Quiz PDF: questions first, then the answer key, then lessons with pages. Mark flagged items.
- Build with HTML + playwright `page.pdf`. Check fonts with `pdffonts` and look at 3 pages.

8. Lessons from the first run

- The user said ~500 MCQs; the file had 375. Count and say so.
- OCR by a small model slips on look-alike words ("লক্ষ্য" vs "লঙ্ঘন") and conjuncts. Spot check and fix.
- Split zips: `zip -s 0` can fail. Concatenate the parts and inflate the entry with python `zlib` (raw deflate).
- Two MCQ styles appeared (red box, white dot). Pen marks by a teacher can disagree with the printed marker. Flag them.
- The book can be wrong or inconsistent (a printed code output that skips a number). Flag, do not silently fix.
- A formula or fact may be in the MCQs but not in the book. Keep the key answer, mark `UNSURE`.
- Artifact pages cannot load fonts from most CDNs: embed fonts as base64. `window.print` is blocked, so ship PDFs as files.

9. The megaprompt (copy, fill the brackets, paste)

```
ROLE
Build my last-minute exam kit. Subject: [SUBJECT, LEVEL]. Exam: ~[N] MCQs tomorrow.
Study time: [H] hours, [C] chapters. Goal: [SCORE]. Accuracy first.

LANGUAGE
All output in [LANGUAGE], in the book's own wording. Keep English technical terms exactly as the
book prints them.

FILES
Book: [PATH]. MCQ bank: [PATH]. Answer marker: [HOW]. Chapter split: [HOW].
Difficulty (1-3): [ch1..chC].

BUDGET AND MODELS (hard rules)
Budget: [$X], stop and ask at 70%. No Opus for OCR/extraction/tagging. Text layer first; Haiku if
OCR is needed.
Sonnet builds chapters, max 6 agents, one per chapter, 5-line replies, agents read instruction files
by path.
Opus only for one short audit of flagged answer-key conflicts. Scripts for everything mechanical.
View images only for doubts and 10 random OCR checks. Commit and push after each stage. Keep
HANDOFF.md.

HOW YOU WORK
Auto-accept edits. Plan the data format once, then build heaviest chapter first. Tell me when each
chapter is ready.
Review only flagged items, max 3 rounds. Report fixes. Mock-test the app before real data.

ACCURACY
Every fact from the book with printed page number. Never invent. Unsure = UNSURE + reason.
State facts with context (true when X, false when Y). Flag wrong answer keys and book disagreements.
Run code questions. Spot check 10 facts per chapter.

STEP 1 MCQ analysis: map each MCQ to chapter + topic; counts per chapter; top 10 repeated topics per
chapter; question types; distractor traps.
```

```
STEP 2 Weights: time = MCQ count x difficulty; show table + reason.
STEP 3 Notes: one section per chapter by weight, built from the book and every MCQ concept. Star
repeated facts, add trap lines, 5-line must-remember box. Zero unsupported MCQs. Reading time =
words/150 per min, target [MIN] minutes.
STEP 4 App (single HTML artifact): [LANGUAGE] UI, embedded font, dark/light, phone friendly. One
card at a time. Lesson after each answer with page.
  Review rule: correct = mastered, never repeat. Wrong = back in 3 cards, correct next time = done.
  Question count selector 10-500 (best N by chapter weight and topic spread). Modes: mix, by
chapter, wrong only, starred only, morning 15 min.
  Settings: theme, 13 accent colors, text size, shuffle, hints, timer, starred only, chapter filter.
Mastery bars, weak spots, confetti (reduced motion safe). Progress in localStorage with try/catch.
STEP 5 PDFs: notes PDF (weighted order, embedded font, page numbers) and quiz PDF (answer key and
lessons separate). Check every page renders.
FINAL CHECK: test app (theme, settings, review rule, count selector, fonts), verify keys, verify
PDFs. Publish the artifact, push the repo, send the PDFs.
```

10. Reuse the reference code (saves the most tokens)

Copy `ict-exam-kit/scripts` into your project. Edit only:

- `build.py`: `DIFF` (difficulty per chapter), `NOTES_MIN`, `QUIZ_MIN`, `PATTERNS` (question types in your language).
- `app_template.html`: UI strings (about 100) and accent names. Ask one Sonnet pass to translate them.
- `CHAPTER_INSTRUCTIONS.md`, `MCQ_INSTRUCTIONS.md`, `REVIEW_INSTRUCTIONS.md`: change the language and style rules.

Run order: extract text, extract MCQs, split by chapter, build chapters (Sonnet), `python3 scripts/build.py`, `node scripts/make_pdf.js ...`, `node scripts/test_app.js ...`, publish.